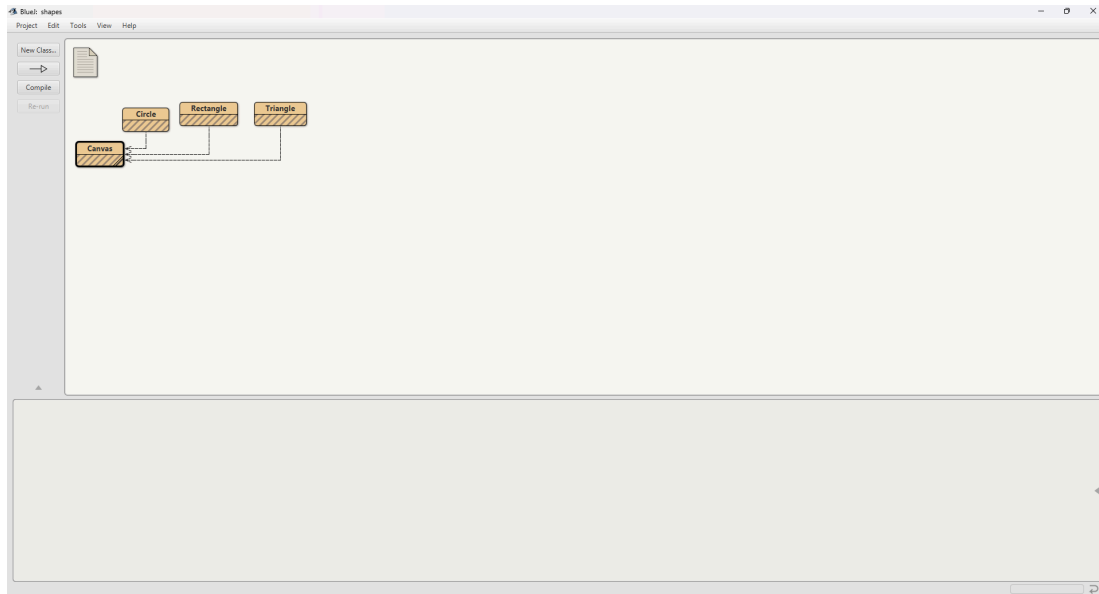


Parte 1 Shapes

A Conociendo el proyecto.

1. El proyecto shapes es una versión modificada de un recurso ofrecido por BlueJ. Para trabajar con él, bajen shapes.zip y ábralo en BlueJ Capturen la pantalla.



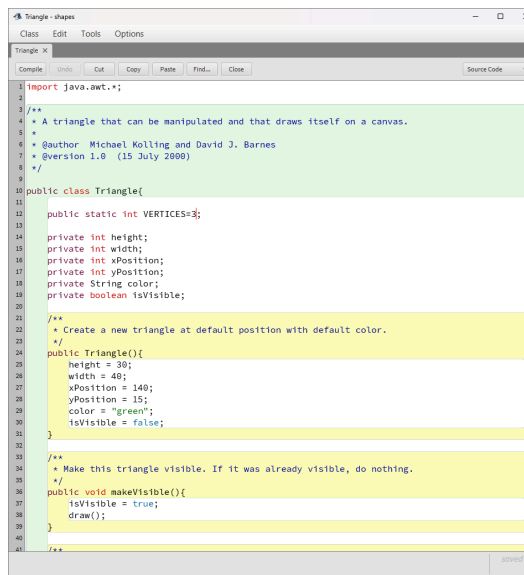
2. El diagrama de clases permite visualizar las clases de un artefacto software y las relaciones entre ellas. Considerando el diagrama de clases de shapes: (a) ¿Qué clases ofrece? (b) ¿Qué relaciones existen entre ellas?
 - a. Ofrece 4 clases incluyendo el canvas
 - b. Las líneas punteadas con flecha desde Circle, Rectangle y Triangle hacia Canvas representan una relación de dependencia.
3. La documentación presenta las clases del proyecto y, en este caso, la especificación de sus componentes públicos. Genere la documentación. De acuerdo con la documentación: (a) ¿Qué clases tiene el paquete shapes? (b) ¿Qué atributos tiene la clase Triangle? (c) ¿Cuántos métodos ofrece la clase Triangle? (d) ¿Qué atributos determinan el tamaño de un Triangle? (e) ¿Cuáles métodos ofrece la clase Triangle para cambiar su tamaño?
 - a. El paquete de shapes tiene las clases Canvas, Circle, Rectangle, Triangle.
 - b. La clase Triangle tiene los atributos: VÉRTICES
 - c. La clase Triangle ofrece 13 métodos
 - d. Los atributos que determinan el tamaño de un Triangle son: height y width
 - e. El método que me ayuda a cambiar el tamaño es public void changeSize(int newHeight, int newWidth),
4. En el código fuente de cada clase está el detalle de la implementación. Revisen el código de la clase Triangle. Con respecto a los atributos: (a) ¿Cuántos atributos realmente tiene? (b)

¿Quiénes pueden usar los atributos públicos?. Con respecto a los métodos: (c) ¿Cuántos métodos tiene en total? (d) ¿Quiénes usan los métodos privados?

- Tiene 7 atributos
- Todas las clases pueden usar los atributos públicos
- Tiene 15 métodos en total
- Los métodos privados son sólo usados por la misma clase

5. Desde el editor, consulte la documentación de Triangle. (a) Capture la pantalla. Comparando la documentación con el código: (b) ¿Qué no se ve en la documentación? (c) ¿por qué debe ser así?

a.



```
import java.awt.*;

/**
 * A triangle that can be manipulated and that draws itself on a canvas.
 *
 * @author Michael Kolling and David J. Barnes
 * @version 1.0 (15 July 2000)
 */
public class Triangle {

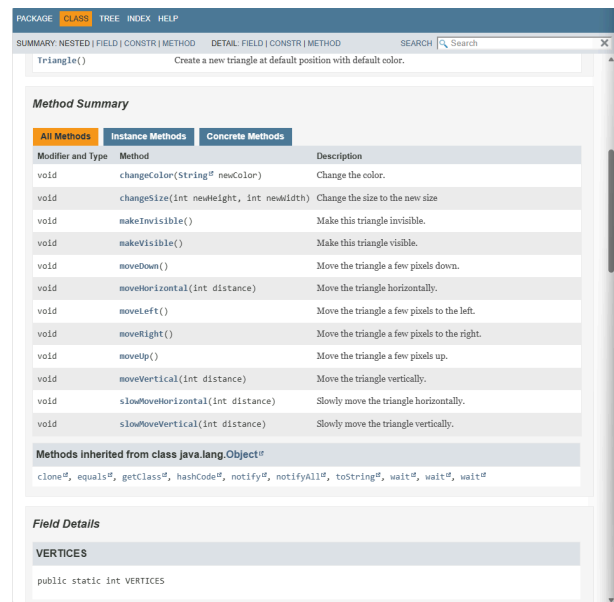
    public static int VERTICES=3;

    private int height;
    private int width;
    private int xPosition;
    private int yPosition;
    private String color;
    private boolean isVisible;

    /**
     * Create a new triangle at default position with default color.
     */
    public Triangle() {
        height = 30;
        width = 40;
        xPosition = 140;
        yPosition = 15;
        color = "green";
        isVisible = false;
    }

    /**
     * Make this triangle visible. If it was already visible, do nothing.
     */
    public void makeVisible() {
        isVisible = true;
        draw();
    }

    /**
     * Draw the triangle on the canvas.
     */
    public void draw() {
        // ...
    }
}
```



PACKAGE CLASS TREE INDEX HELP

SUMMARY NESTED | FIELD | CONSTR | METHOD DETAIL FIELD | CONSTR | METHOD SEARCH

Triangle()

Create a new triangle at default position with default color.

Method Summary

Modifier and Type	Method	Description
void	changeColor(String [®] newColor)	Change the color.
void	changeSize(int newHeight, int newWidth)	Change the size to the new size
void	makeInvisible()	Make this triangle invisible.
void	makeVisible()	Make this triangle visible.
void	moveDown()	Move the triangle a few pixels down.
void	moveHorizontal(int distance)	Move the triangle horizontally.
void	moveLeft()	Move the triangle a few pixels to the left.
void	moveRight()	Move the triangle a few pixels to the right.
void	moveUp()	Move the triangle a few pixels up.
void	moveVertical(int distance)	Move the triangle vertically.
void	slowMoveHorizontal(int distance)	Slowly move the triangle horizontally.
void	slowMoveVertical(int distance)	Slowly move the triangle vertically.

Methods inherited from class java.lang.Object[®]

clone[®], equals[®], getClass[®], hashCode[®], notify[®], notifyAll[®], toString[®], wait[®], wait[®], wait[®]

Field Details

VERTICES

public static int VERTICES

b. Lo que no se ve en la documentación es:

- El código fuente o implementación de los métodos
- Los atributos y métodos que no se ven en la documentación son los que son private.

c. Porque la documentación está diseñada para mostrar únicamente la interfaz pública de la clase, es decir los elementos que otras clases pueden utilizar. Los atributos y métodos privados forman parte de la implementación interna y están ocultos para cumplir el principio de encapsulamiento de la programación orientada a objetos.

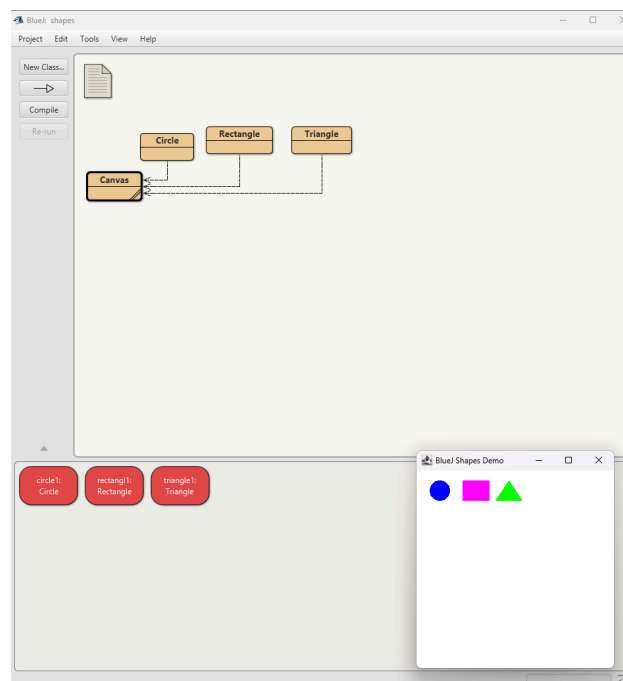
6. ¿Cuál dirían es el propósito del proyecto shapes?

a. Demostrar de forma muy simple las características básicas de los objetos, permitiendo crear figuras (cuadrados, triángulos, círculos) que se dibujan en una ventana ("canvas") y luego pueden manipularse interactivamente (cambiar posición, tamaño y color), es decir, es un proyecto pensado para que nosotros como estudiantes veamos "en vivo" qué es una clase, qué es un objeto y cómo interactúan, antes de pasar a proyectos más complejos como "picture" (que reutiliza estas figuras para dibujar imágenes).

B Manipulando objetos

1. Creen un objeto de cada una de las clases que lo permitan y haganlos visibles. (a) ¿Cuántas clases tiene el proyecto? (b) ¿Cuántos objetos crearon? (c) ¿Qué clase se crea de forma diferente? ¿Por qué? (d) Capturen la pantalla.

- Tiene 4 clases
- Se crearon 3 objetos un Circle, un Rectangle, un Triangle,
- La clase Canvas porque no se instancia con `new Canvas()`. El lienzo se crea automáticamente cuando alguna figura se hace visible ya que internamente las figuras llaman al método: `Canvas.getCanvas()`; Este método devuelve una única instancia de Canvas
- d.



2. Inspeccionen los creadores de cada una de las clases. (a) ¿Cuál es la principal diferencia entre ellos? (b) ¿Qué se busca con la clase que tiene el creador diferente?

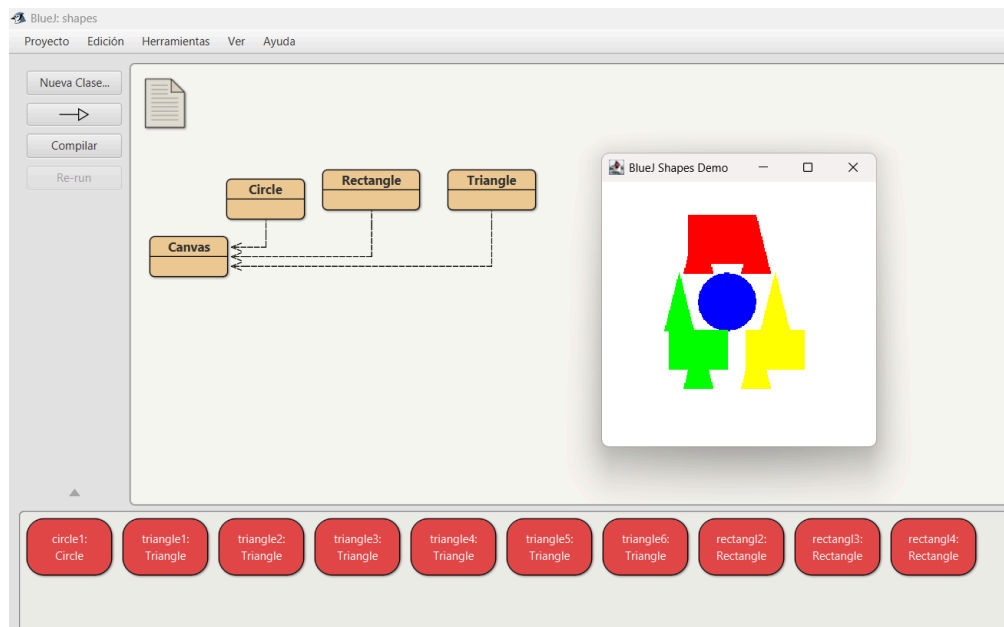
- La principal diferencia es que las clases Circle, Rectangle y Triangle tienen constructores públicos por lo que se pueden crear directamente utilizando el operador `new`. En cambio la clase Canvas se crea mediante un método privado
- Se busca que otro canvas no pueda ser creado por otra clase, es decir, para que solo exista un solo canvas

3. Construyan, con shapes sin escribir código, una propuesta de la imagen del logo de su navegador favorito. (a) ¿Cuántas y cuáles clases se necesitan? (b) ¿Cuántos objetos se usan en total? (c) Capturen la pantalla. (d) Incluyan el logo original

- Use las cuatro clases las cuales fueron Canvas, Circle, Rectangle, Triangle.

b. Se usaron 10 objetos

c.



d.



C Detallando una clase y explorando sus objetos

1. En el código de la clase Triangle, revise el atributo VÉRTICES. (a) ¿Qué significa que sea public? (b) ¿Qué significa que sea static? (c) ¿Debería ser final? ¿Por qué? (d) Realicen los cambios necesarios.

- Significa que cualquier otra clase puede acceder directamente al atributo VÉRTICES.
- Quiere decir que el atributo pertenece a la clase y no a cada instancia creada de la clase
- Si, porque al ser final nos dice que es un valor que siempre va hacer ese es decir que solo se puede asignar una vez y luego no se puede cambiar. En nuestro caso lo que

queremos es un triángulo y este siempre va a tener tres vértices Al usar final, el valor queda como una constante y no puede modificarse después de inicializarse.

d. `public static final int VERTICES=3;`

2. En el código de la clase Triangle revisen el detalle del tipo del atributo height. (a) ¿Qué se está indicando al decir que es int? (b) Si fuera byte, ¿cuál sería el área más grande posible? (c) y ¿si fuera long? (d) ¿qué restricción adicional debería tener este atributo? (e) Refactoricen el código considerando (d).

- a. int indica que height es un número entero de 32 bits. Puede almacenar valores desde: -2,147,483,648 y máximo de 2,147,483,647 esto es la altura del triángulo en píxeles.
- b. Un byte en Java puede almacenar desde -128 hasta 127 y para calcular el área se asumirá que width también es byte entonces será

$$A = (127 * 127)/2 = 8064,5 \text{ pixeles}^2$$

- c. long es un entero de 64 bits con signo, por lo que puede almacenar desde: -9.223.372.036.854.775.808 hasta 9.223.372.036.854.775.807. Esto permitiría representar alturas muchísimo mayores que con int.

$$A = (9.223.372.036.854.775.808 * 9.223.372.036.854.775.808)/2 = 4.25 * 10^{37}$$

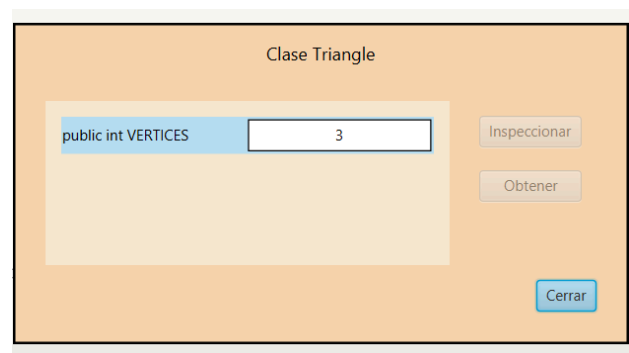
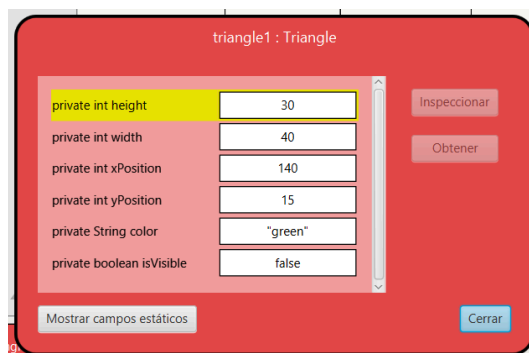
- d. La altura no debería poder ser negativa es decir: *height* >= 0
- e. `if(newHeight >= 0)`

3. Si creamos 100 triángulos, (a) ¿cuántos VERTICES y cuántos height tendríamos? (b) Justifique.

- a. Tendríamos VERTICES = 1 y height = 100
- b. VERTICES pertenece a la clase Triangle no a cada objeto por ser static. Por eso aunque creamos 100 triángulos, existe un solo VERTICES, compartido por todos, cuyo valor es 3 en cambio height pertenece a cada objeto individual por lo tanto cada uno de los 100 triángulos tiene su propio height.

4. Inspeccionen el estado del objeto :Triangle. (a) Capturen la pantalla. (b) ¿Cuál es el valor de height inicial? (c) ¿Cuál es el valor de VERTICES?

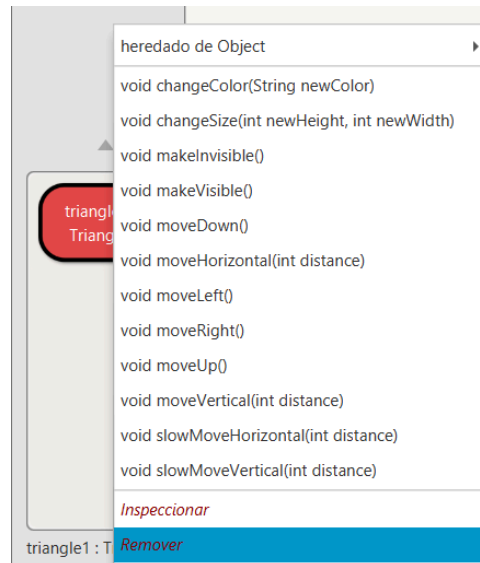
a.



- b. El valor inicial de height es 30
- c. El valor de VERTICES es 3

5. Inspeccionen el comportamiento que ofrece el objeto :Triangle. (a) Capturen la pantalla. (b) ¿Por qué no aparecen todos los métodos que están en el código?

a.



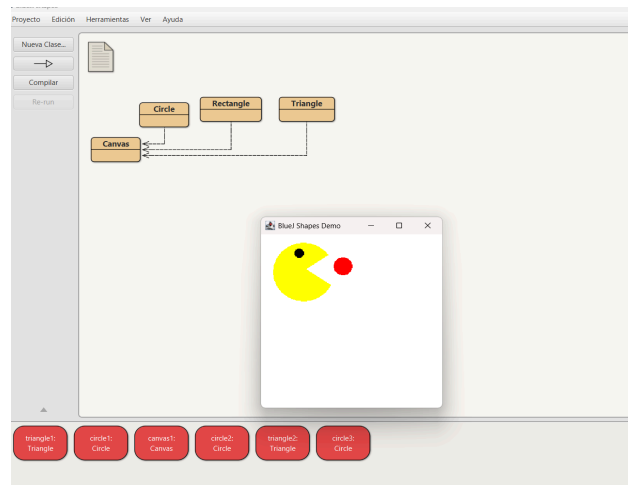
- b. Porque en el código hay dos métodos declarados como private
 private void draw()
 private void erase()

D Analizando y escribiendo código

```
Circle face;
Circle eye;
Triangle mouthUp;
Triangle mouthDown;
//1
face = new Circle();
face.changeColor("yellow");
face.changeSize(100);
//2
eye = new Circle();
eye.changeColor("black");
eye.changeSize(15);
eye.moveVertical(10);
eye.moveHorizontal(35);
eye.makeVisible();
//3
```

```
mouthUp = new Triangle();
mouthUp.changeColor("white");
mouthUp.changeSize(30, 90);
mouthUp.moveHorizontal(-20);
mouthUp.moveVertical(15);
mouthUp.makeVisible();
//4
mouthDown = new Triangle();
mouthDown.changeColor("white");
mouthDown.changeSize(-30, 90);
mouthDown.moveHorizontal(-20);
mouthDown.moveVertical(75);
mouthDown.makeVisible();
//5
Circle food=eye;
food.changeColor("red");
food.moveHorizontal(100);
food.moveVertical(25);
food.makeVisible();
//6
```

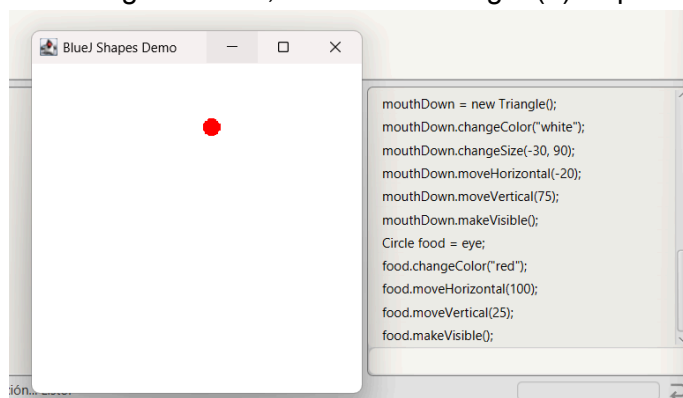
1. Lean el código anterior. (a) ¿Cuál creen que es la figura resultante? (b) Píntenla.
 - a. La figura resultante del código anterior es un pac-man comiendo uno de los puntos rojos característicos del juego.
 - b.



2. Para cada punto señalado: (a) ¿cuántas variables existen? (b) ¿cuántos objetos existen? (c) ¿cuántos objetos se ven? (no cuenten ni los objetos String ni el objeto Canvas) (3 respuestas en cada punto)

Número del paso	a: Variables	b: Objetos existentes	c: Objetos Visibles
//1	4	0	0
//2	4	1	0
//3	4	2	1
//4	4	3	2
//5	4	4	3
//6	5	4	4

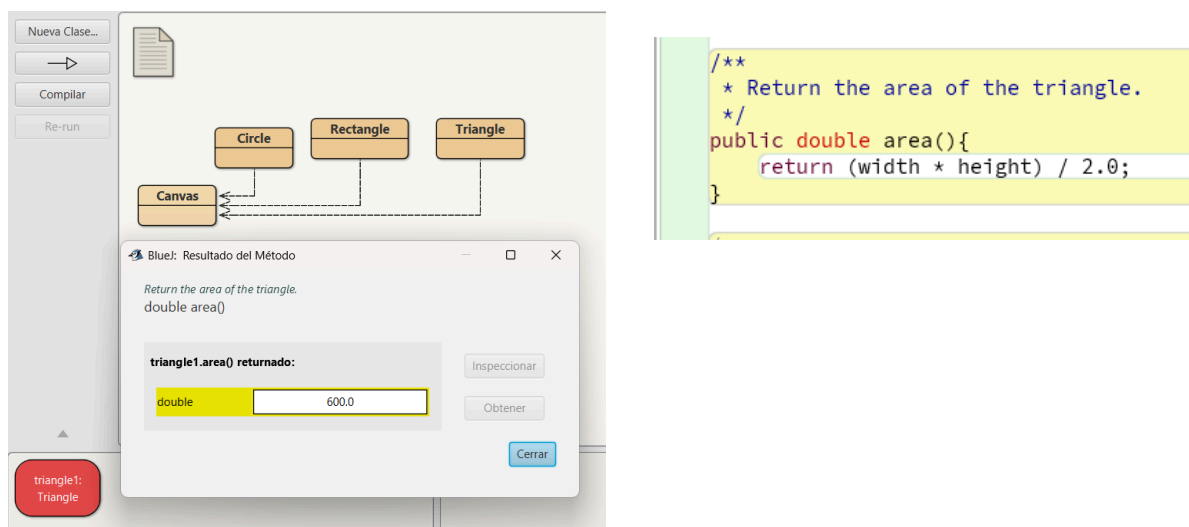
3. Habiliten la ventana de código en línea, escriban el código. (a) Capturen la pantalla.



4. Compare la figura pintada en 1. con la figura capturada en 3.: (a) ¿son iguales? (b) ¿por qué?
- No son iguales
 - Porque el código tiene unos errores entonces cuando se hace a mano estos errores no se ven reflejados pero al hacerlo en código ya los errores no muestran la figura que se esperaba
5. (a) ¿Qué errores hacen que la figura no quede bien pintada? (b) ¿Qué figura es?
- Los errores que hacen que la figura no quede bien es:
 - face nunca recibe makeVisible()
 - mouthDown.changeSize(-30, 90) falla silenciosamente porque changeSize solo actúa si newHeight >= 0 por lo tanto mouthDown se queda con el tamaño por defecto (30×40)
 - Circle food = eye no crea un objeto nuevo simplemente es otro nombre para la variable que en vez de ser comida y el ojo lo que va hacer es un círculo que termina rojo y desplazado lejos de su posición original.
 - Lo que aparece en el Canvas es un círculo rojo + dos triángulos blancos desiguales

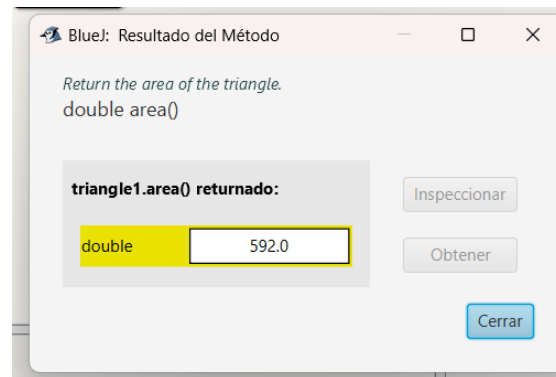
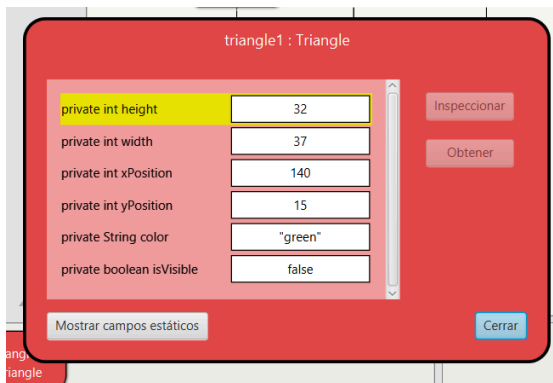
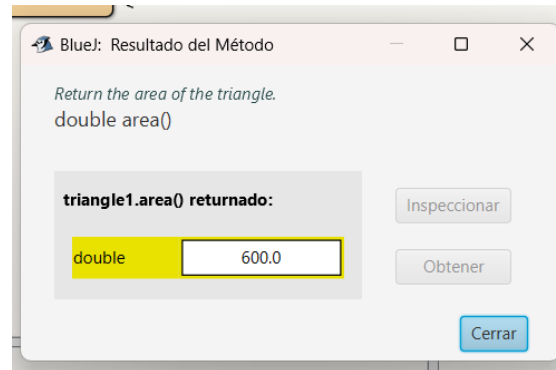
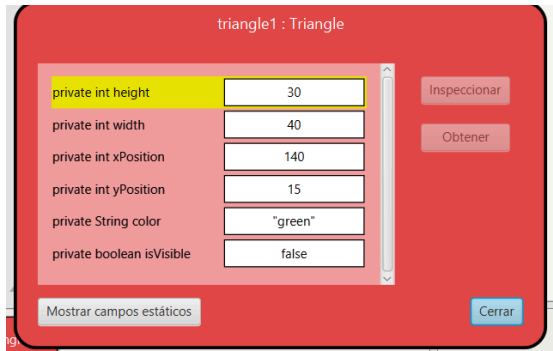
E Extendiendo una clase. Triangle.

1. Desarrollen en Triangle el método área() (retorna el área). ¡Pruébenlo! Capturen una pantalla.



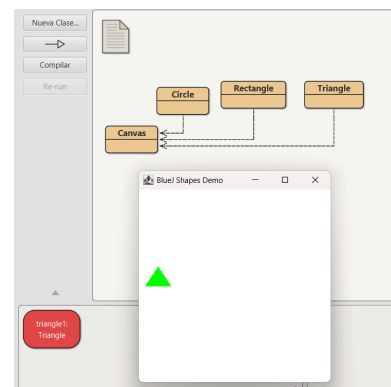
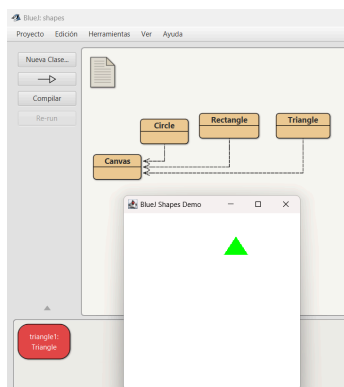
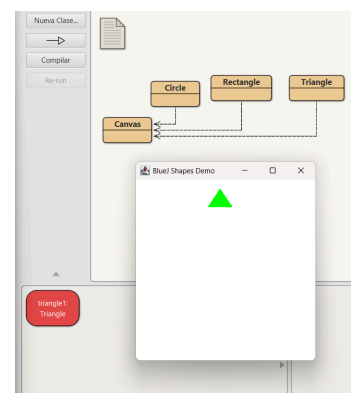
2. Desarrollen en Triangle el método equilateral() (lo convierte en un triángulo equilátero de aproximadamente la misma área) ¡Pruébenlo! Capturen dos pantallas.

```
/**
 * Convert this triangle into an equilateral triangle with
 */
public void equilateral(){
    double currentArea = area();
    double side = Math.sqrt(currentArea * 4 / Math.sqrt(3));
    int newWidth = (int) Math.round(side);
    int newHeight = (int) Math.round(side * Math.sqrt(3) / 2);
    changeSize(newHeight, newWidth);
}
```



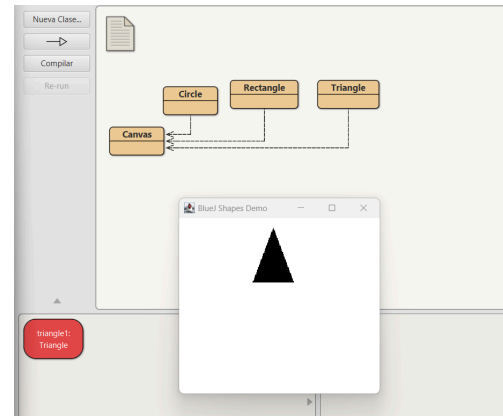
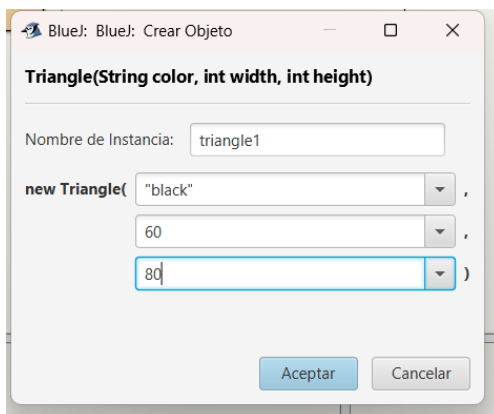
3. Desarrollen en Triangle el método walk (times: int) (va moviendose cayendo hacia la derecha (positivo) o izquierda (negativo) abs(times) veces). ¡Pruébenlo! Capturen tres pantallas.

```
/**
 * Move the triangle to the right (positive) or left (negative) as it falls.
 */
public void walk(int times){
    int steps = Math.abs(times);
    int direction;
    if (times >= 0) {
        direction = 1;
    } else {
        direction = -1;
    }
    for (int i = 0; i < steps; i++){
        moveVertical(5);
        moveHorizontal(10 * direction);
    }
}
```



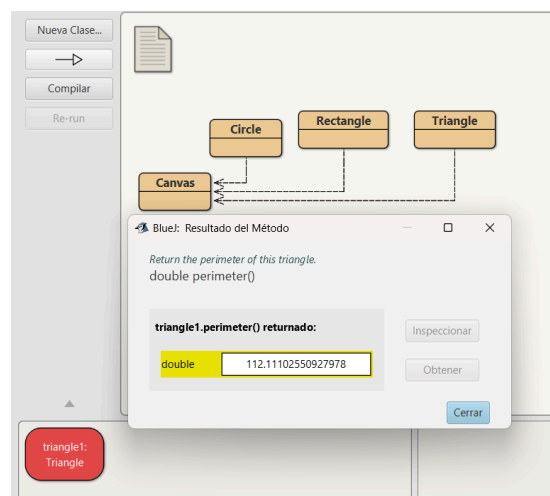
4. Desarrollen en Triangle un nuevo creador que permita crearlo dado su color, su ancho y su alto. ¡Pruébenlo! Capturen una pantalla.

```
public Triangle(String color, int width, int height){  
    this.height = height;  
    this.width = width;  
    xPosition = 140;  
    yPosition = 15;  
    this.color = color;  
    isVisible = false;  
}
```



5. Propongan un nuevo método para esta clase. Desarrollen y prueben el método.

```
/**  
 * Return the perimeter of this triangle.  
 */  
public double perimeter(){  
    double side = Math.sqrt(Math.pow(width / 2.0, 2) + Math.pow(height, 2));  
    return width + (2 * side);  
}
```



6. Generen nuevamente la documentación y revisen la información de estos nuevos métodos. Capturen la pantalla.

Constructor Summary	
Constructors	
Constructor	Description
<code>Triangle()</code>	Create a new triangle at default position with default color.
<code>Triangle(String^c color, int width, int height)</code>	Create a new triangle at default position with the specified color,width, and height.

area	[Show source in BlueJ]
<pre>public double area()</pre> Return the area of the triangle. Returns: the area of the triangle	
equilateral	[Show source in BlueJ]
<pre>public void equilateral()</pre> Convert this triangle into an equilateral triangle with	
walk	[Show source in BlueJ]
<pre>public void walk(int times)</pre> Move the triangle to the right (positive) or left (negative) as it falls. Parameters: times - the number of steps to move; positive values move right, negative values move left	
perimeter	[Show source in BlueJ]
<pre>public double perimeter()</pre> Return the perimeter of this triangle. Returns: the perimeter of the triangle	

PARTE 3

A. Creando una nueva clase: Robot

1. Inicie la construcción únicamente con los atributos. Justifique su selección. Agregue pantallazo con los atributos.

- x y y: son variables de tipo int que representan las coordenadas de la posición actual del robot dentro del laberinto. Se necesitan porque el robot debe conocer su ubicación y el método `coordinates()` debe poder retornar su posición {x,y}.
- direction: es una variable de tipo char que representa la dirección hacia la que está mirando el robot. Puede tomar los valores correspondientes a Norte (N), Sur (S), Este (E) u Oeste (W).

- body: es un objeto de tipo Rectangle que representa visualmente el cuerpo del robot.
- eye1 y eye2: son objetos de tipo Circle que representan los dos ojos del robot y van a estar apuntando la dirección del robot.
- mouth: es un objeto de tipo Rectangle que representa visualmente la boca del robot.
- foot1 y foot2: son objetos de tipo Rectangle que representan los dos pies del robot que estarán al otro lado de los ojos.

```
/**
 * Representa un robot que puede desplazarse por un laberinto,
 * cambiar de dirección y mostrar su posición y apariencia.
 * El robot está compuesto visualmente por un cuerpo, dos ojos,
 * una boca y dos pies.
 *
 * @author MoralesS - RojasH
 * @version 1.0 (08 August 2026)
 */
public class Robot
{
    private int x;
    private int y;
    private char direction;
    private boolean ok;
    // Diseño del robot
    private Rectangle body;
    private Circle eye1;
    private Circle eye2;
    private Rectangle mouth;
    private Rectangle foot1;
    private Rectangle foot2;
}
```

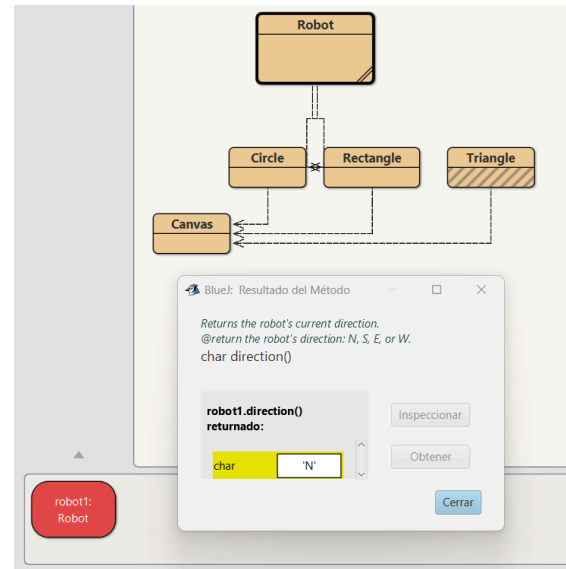
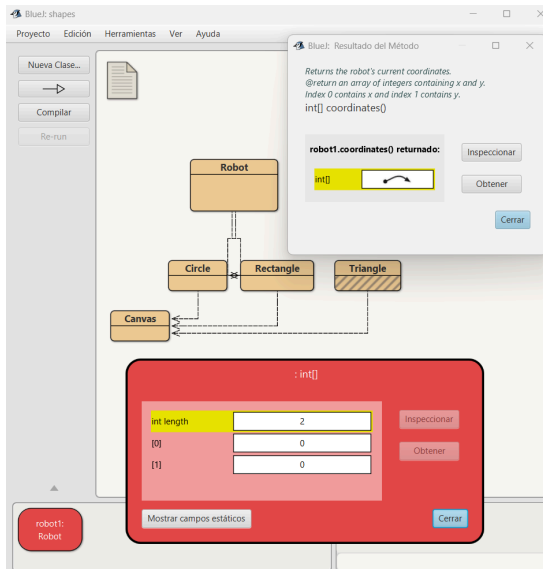
2. Desarrollen la clase considerando los 3 mini-ciclos. No olviden la documentación. Al final de cada mini-ciclo realicen dos pruebas indicando su propósito. Capturen las pantallas relevantes.

- Ciclo 1.

```
/**
 * Create a new robot with default values
 * at position (0, 0) and facing north.
 */
public Robot(){
    x = 0;
    y = 0;
    direction = 'N';
}

/**
 * Returns the robot's current coordinates.
 * @return an array of integers containing x and y.
 * Index 0 contains x and index 1 contains y.
 */
public int[] coordinates(){
    return new int[] {x,y};
}

/**
 * Returns the robot's current direction.
 * @return the robot's direction: N, S, E, or W.
 */
public char direction(){
    return direction;
}
```

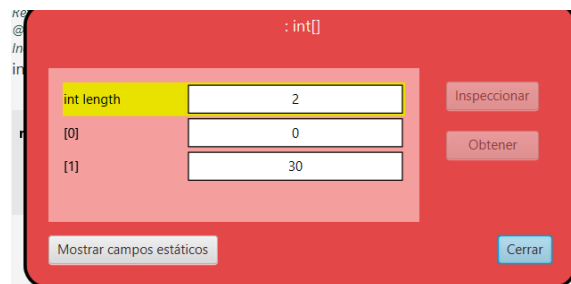
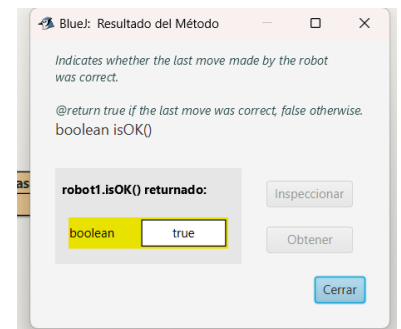
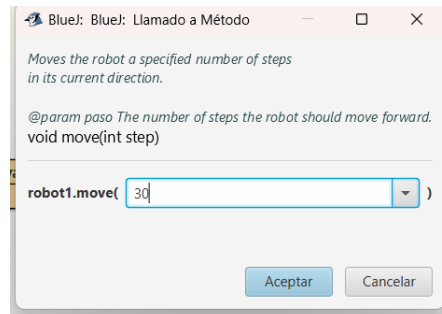


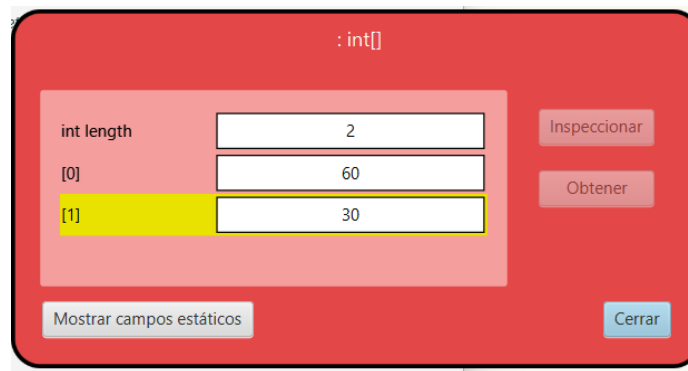
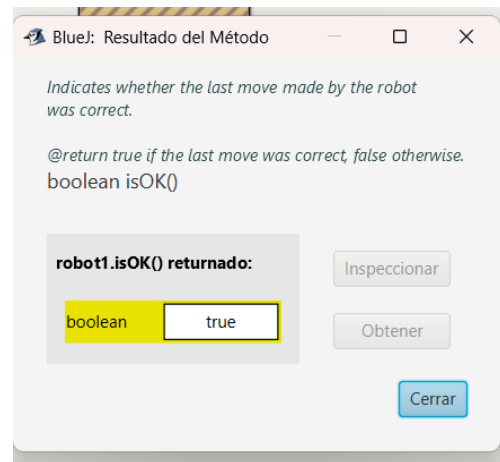
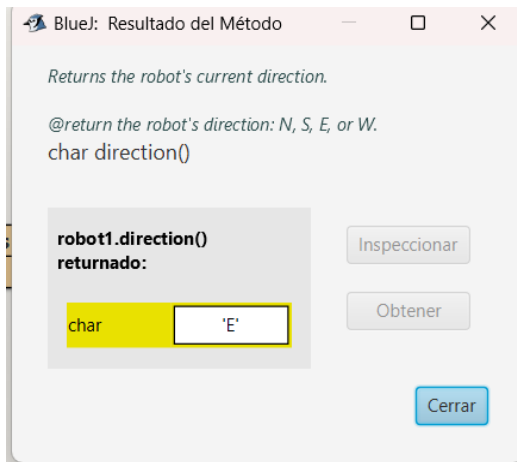
• Ciclo 2

```

54  /**
55   * Moves the robot a specified number of steps
56   * in its current direction.
57   *
58   * @param paso The number of steps the robot should move forward.
59   */
60  public void move(int step){
61      if (direction == 'N') {
62          y = y + step;
63      }
64      else if (direction == 'S') {
65          y = y - step;
66      }
67      else if (direction == 'E') {
68          x = x + step;
69      }
70      else if (direction == 'W') {
71          x = x - step;
72      }
73  }
74
75  /**
76   * Rotates the robot 90 degrees to the right.
77   * The order of directions is N, E, S, W.
78   */
79  public void turn(){
80      if (direction == 'N'){
81          direction = 'E';
82      }
83      else if (direction == 'E'){
84          direction = 'S';
85      }
86      else if (direction == 'S') {
87          direction = 'W';
88      }
89      else if (direction == 'W') {
90          direction = 'N';
91      }
92  }
93
94  /**
95   * Indicates whether the last move made by the robot
96   * was correct.
97   *
98   * @return true if the last move was correct, false otherwise.
99   */
100 public boolean isOK(){
101     return ok;
102 }

```





• Ciclo 3

```

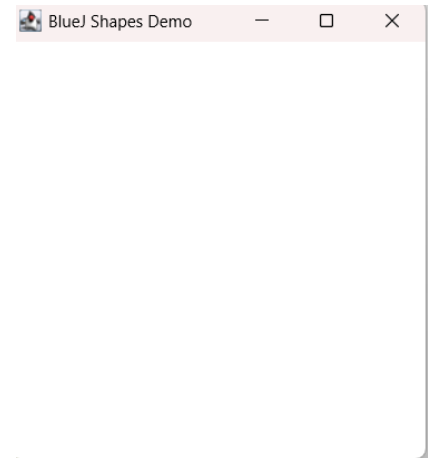
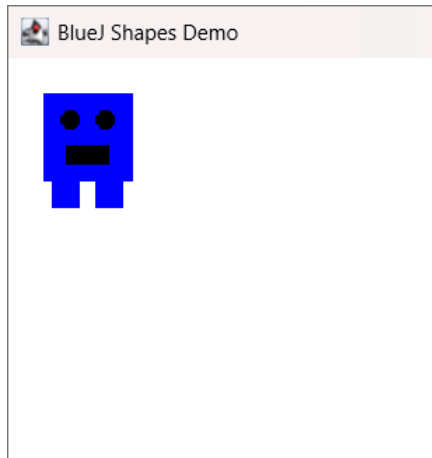
137 /**
138  * Crea un nuevo robot con los valores predeterminados.
139  *
140  * El robot comienza en la posición (0, 0), orientado
141  * hacia el norte y con un estado correcto.
142  * También crea y organiza visualmente todas las partes
143  * que componen el robot.
144  */
145 public Robot(){
146     x = 0;
147     y = 0;
148     direction = 'N';
149     ok = true;
150
151     body = new Rectangle();
152     eye1 = new Circle();
153     eye2 = new Circle();
154     mouth = new Rectangle();
155     foot1 = new Rectangle();
156     foot2 = new Rectangle();
157     // Organizar las partes del robot
158     eye1.setColor("black");
159     eye2.setColor("black");
160     body.setColor("blue");
161     foot1.setColor("blue");
162     foot2.setColor("blue");
163     mouth.setColor("black");
164
165     body.setSize(50, 50);
166     eye1.setSize(10);
167     eye2.setSize(10);
168     mouth.setSize(10, 24);
169     foot1.setSize(15, 15);
170     foot2.setSize(15, 15);
171
172     eye1.moveHorizontal(10);
173     eye1.moveVertical(10);
174     eye2.moveHorizontal(30);
175     eye2.moveVertical(10);
176     mouth.moveHorizontal(15);
177     mouth.moveVertical(30);
178     foot1.moveHorizontal(5);
179     foot1.moveVertical(50);
180     foot2.moveHorizontal(30);
181     foot2.moveVertical(50);
182
183     anchorToGrid();
184 }

```

```

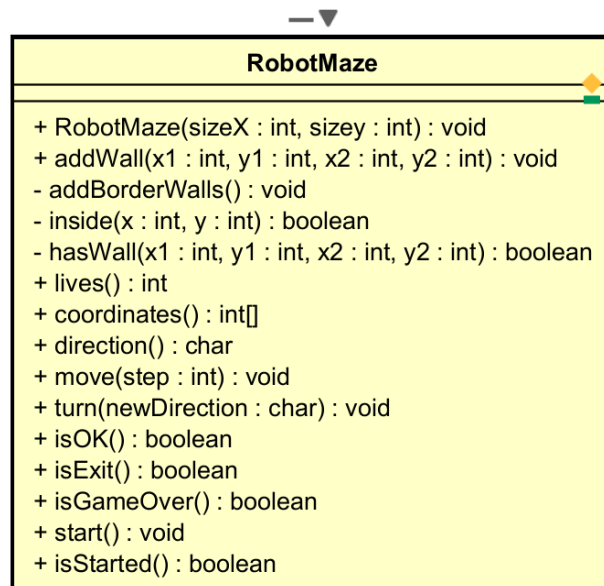
137 /**
138  * Makes the robot visible by showing its body,
139  * eyes, mouth, and feet.
140  */
141 public void makeVisible()
142 {
143     body.makeVisible();
144     eye1.makeVisible();
145     eye2.makeVisible();
146     mouth.makeVisible();
147     foot1.makeVisible();
148     foot2.makeVisible();
149 }
150
151 /**
152  * Renders the robot invisible by concealing its body,
153  * eyes, mouth, and feet.
154  */
155 public void makeInvisible()
156 {
157     body.makeInvisible();
158     eye1.makeInvisible();
159     eye2.makeInvisible();
160     mouth.makeInvisible();
161     foot1.makeInvisible();
162     foot2.makeInvisible();
163 }
164 }
165

```



B Definiendo y creando una nueva clase.

1. Diseñen la clase, es decir, definan los métodos que debe ofrecer.



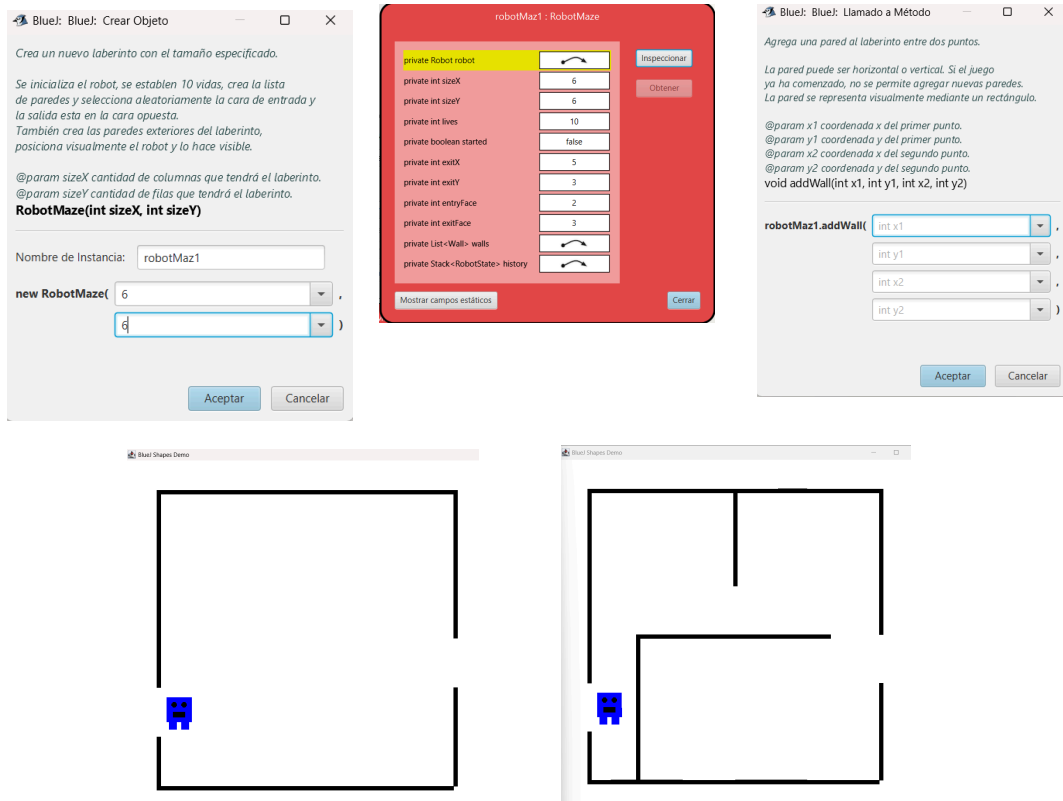
2. Planifiquen la construcción considerando algunos mini-ciclos.

- CICLO 1 - Creación y construcción del laberinto: En este ciclo se desarrolla todo lo relacionado con crear el tablero y las paredes. Se tiene como objetivo Tener un laberinto creado, con sus límites y paredes, y poder comprobar si una posición o pared es válida
 - RobotMaze(sizeX : int, sizeY : int)
 - addWall(x1 : int, y1 : int, x2 : int, y2 : int)
 - addBorderWalls()
 - inside(x : int, y : int)
 - hasWall(x1 : int, y1 : int, x2 : int, y2 : int)
- CICLO 2 - Movimiento y control del robot: Aquí se desarrolla la interacción del robot con el laberinto. Lo que se quiere es que el robot se mueva, cambie de dirección y detecte si su movimiento es correcto, teniendo en cuenta las paredes.

- `coordinates() : int[]`
- `direction() : char`
- `move(step : int)`
- `turn(newDirection : char)`
- `isOK() : boolean`
- CICLO 3. Control de la partida. Este ciclo se enfoca en iniciar, controlar y finalizar el juego. es decir cuándo comienza, cuántas vidas quedan, si el robot llegó a la salida y si el juego terminó.
 - `lives() : int`
 - `isExit() : boolean`
 - `isGameOver() : boolean`
 - `start() : void`
 - `isStarted() : boolean`

3. Implementan la clase. Al final de cada mini-ciclo realicen una prueba indicando su propósito. Capturen las pantallas relevantes

- Ciclo 1: Comprobar que el tablero se construye correctamente, verificando que se genere un tablero de tamaño 6×6, que los bordes se creen automáticamente dejando de manera aleatoria los huecos correspondientes a la entrada y salida, y que el robot se ubique correctamente en la entrada. Además, se verifica que sea posible crear y agregar tres paredes utilizando las coordenadas x1, y1, x2, y2.



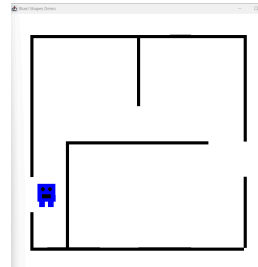
- Ciclo 2: Comprobar el correcto funcionamiento del movimiento y cambio de dirección del robot. Primero, se verifica que el robot pueda desplazarse 3 posiciones hacia el Norte (N) mediante `move(3)`, pasando de la coordenada (0,4) a (0,1). Luego, se comprueba que el robot pueda cambiar correctamente su orientación mediante un giro, pasando de 'N' (Norte) a 'E' (Este).

: int[]

int length	2
[0]	0
[1]	4

Mostrar campos estáticos

Inspeccionar Obtener Cerrar



Blue: BlueJ: Llamado a Método

Mueve el robot una cantidad determinada de pasos.

El método comprueba cada paso para determinar si el robot permanece dentro del laberinto y si existe una pared. Si el robot choca contra una pared o sale del laberinto, pierde una vida y el movimiento se marca como incorrecto.

Si el movimiento es válido, el robot avanza la cantidad de pasos permitidos.

@param step cantidad de pasos que debe intentar realizar el robot. Un valor negativo permite desplazarse en sentido contrario.

void move(int step)

robotMaz1.move(3)

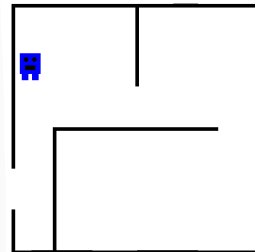
Aceptar Cancelar

: int[]

int length	2
[0]	0
[1]	1

Mostrar campos estáticos

Inspeccionar Obtener Cerrar



Blue: Resultado del Método

Obtiene la dirección actual del robot.

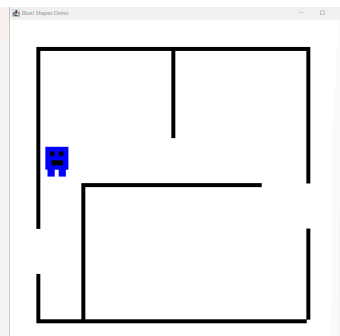
@return dirección actual del robot: N, S, E o W.

char direction()

robotMaz1.direction() retornado:

char 'N'

Inspeccionar Obtener Cerrar



Blue: BlueJ: Llamado a Método

Cambia la dirección del robot.

@param newDirection nueva dirección del robot: 'N', 'S', 'E' o 'W'.

void turn(char newDirection)

robotMaz1.turn('E')

Aceptar Cancelar

Blue: Resultado del Método

Obtiene la dirección actual del robot.

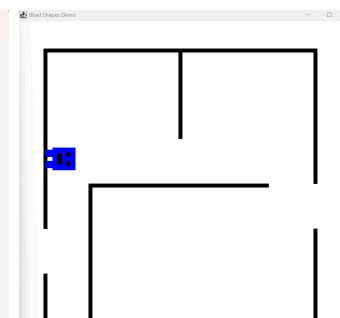
@return dirección actual del robot: N, S, E o W.

char direction()

robotMaz1.direction() retornado:

char 'E'

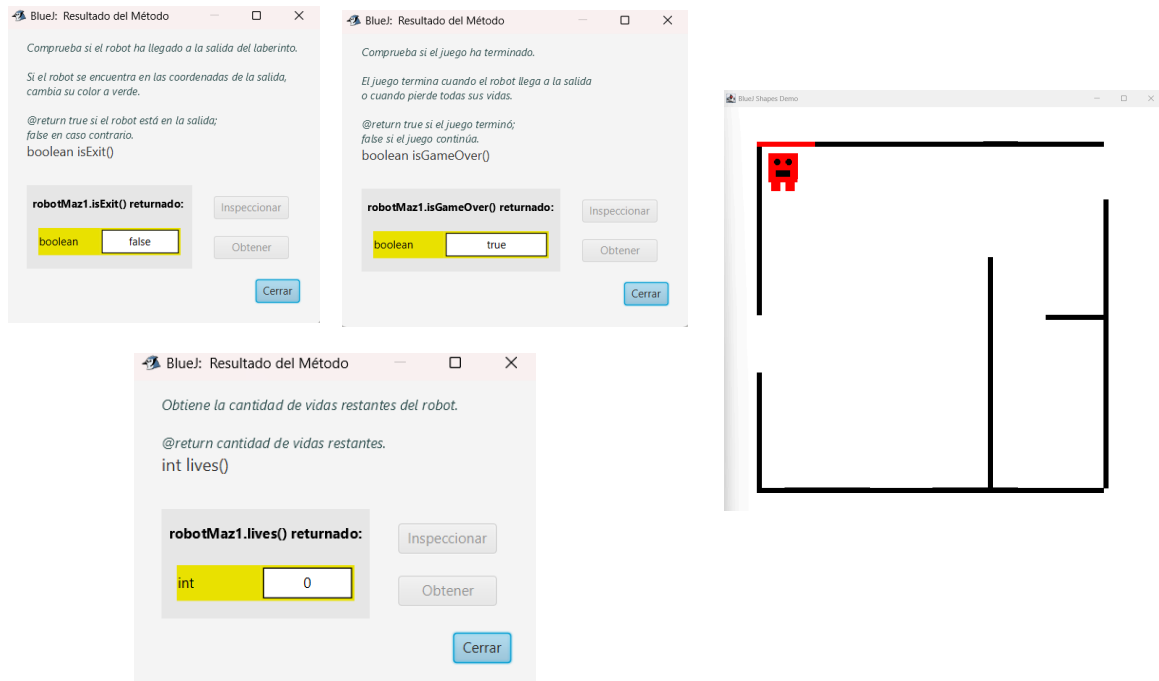
Inspeccionar Obtener Cerrar



- Ciclo 3:

Comprobar el correcto funcionamiento del estado de la partida, verificando las vidas del robot, el inicio del juego, la llegada a la salida y la finalización de la partida tanto en una situación de victoria como de derrota. En la primera prueba, al iniciar el juego, se verifica que el robot cuenta con 10 vidas, que la partida ya comenzó (isStarted = true), que aún no ha llegado a la salida (isExit = false) y que el juego todavía no ha terminado (isGameOver = false). En la segunda prueba, después de realizar varios movimientos, el robot llega a la salida. Conserva sus 10 vidas, por lo que se comprueba que isExit cambia de false a true y isGameOver también cambia a true, indicando que el jugador ganó. Finalmente, en la tercera prueba, el robot pierde todas sus vidas al chocar contra una pared. Por esta razón, lives queda en 0, isExit permanece en false y isGameOver cambia a true, indicando que el jugador perdió la partida.





4. Indiquen las extensiones necesarias para reutilizar la clase Robot y el paquete shapes. Expliquen.

Para reutilizar la clase Robot y los elementos del paquete shapes en RobotMaze, fue necesario realizar varias extensiones y modificaciones para adaptarlos al funcionamiento del tablero.

- shapes: Fue necesario reorganizar los elementos gráficos y establecer el origen en (0,0) para facilitar su ubicación dentro del tablero. También se realizaron ajustes para que las figuras pudieran moverse y rotar correctamente según las posiciones del robot.
- Robot:
 - Variables de posición relativa: permiten conservar la posición de las piezas del robot y mantenerlas correctamente ubicadas al realizar giros.
 - CELL_PADDING: constante utilizada para dejar un pequeño espacio entre el robot y los bordes de las casillas, evitando que quede completamente pegado.
 - anchorToGrid: permite ajustar y ubicar las piezas del robot dentro de la cuadrícula.
 - moveOrigin: permite reubicar las piezas del robot desde su origen hasta la entrada del tablero, que se genera aleatoriamente.
 - applyOrientation: permite rotar correctamente las piezas del robot cuando cambia de dirección.

- `move`: permite realizar desplazamientos horizontales y verticales mediante `dx` y `dy`. Estos valores se multiplican por 100, ya que cada casilla del tablero representa 100 píxeles.
- `setPosition`: actualiza las coordenadas del robot cuando cambia de posición.
- `changeColor`: permite cambiar el color del robot para representar diferentes estados, como cuando el jugador gana o pierde.
- Clase `Wall`: Además, fue necesario crear una nueva clase llamada `Wall` para representar las paredes del laberinto. Esta clase almacena las coordenadas `x1`, `y1`, `x2`, `y2` que indican dónde comienza y termina cada pared, y contiene un objeto `Rectangle` encargado de representar gráficamente la pared. También permite obtener las coordenadas de la pared y el rectángulo correspondiente.

C BONO. Nuevos requisitos funcionales.

1. Diseñen, es decir, definan los métodos que debe ofrecer.

a. Hacer un buen movimiento (la máquina decide).

Método: `goodMove()` Permitir que la máquina decida automáticamente cuál es el mejor movimiento que puede realizar el robot.

Qué debe hacer:

- Revisar las cuatro direcciones posibles: N, S, E y W.
- Comprobar cuáles movimientos son válidos
- Calcular cuál de esas posiciones queda más cerca de la salida.
- Elegir la dirección que permita reducir más esa distancia.
- Realizar un movimiento de una casilla.

Esto corresponde a la estrategia Greedy, utilizando la distancia Manhattan para determinar cuál movimiento acerca más al robot a la salida. En cada movimiento la máquina selecciona la opción válida que tenga menor distancia hacia la salida. Sin embargo esta estrategia no garantiza que siempre encuentre el mejor camino, ya que puede ocurrir que dos o más movimientos tengan la misma distancia Manhattan. En ese caso, se seleccionará una de las opciones disponibles según el orden en que son evaluadas, por lo que el robot podría tomar un camino diferente aunque las opciones sean igualmente cercanas a la salida o que se quede en bucle en esas dos casillas.

b. Deshacer el último movimiento.

Método: `undo()` Permitir que el usuario deshaga el último movimiento realizado por el robot y lo devuelva a la posición y dirección que tenía anteriormente.

Qué debe hacer:

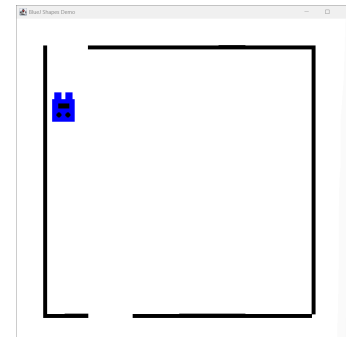
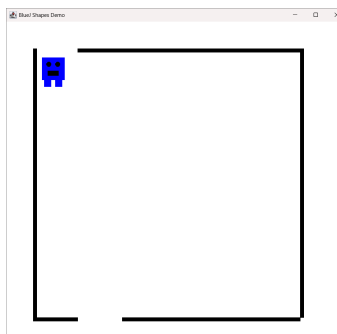
- Guardar los estados anteriores del robot para poder recuperarlos.
- Al solicitar undo(), recuperar el último estado almacenado.
- Restaurar la posición y la dirección anteriores.
- El deshacer debe considerarse como un movimiento

2. Implementen los nuevos métodos. Al final de cada método realicen una prueba indicando su propósito. Capturen las pantallas relevantes.

```

418 //
419 private int manhattanDistance(int x1, int y1, int x2, int y2)
420 {
421     return Math.abs(x1 - x2) + Math.abs(y1 - y2);
422 }
423
424 /**
425  * Realiza el mejor movimiento posible utilizando una estrategia Greedy.
426  *
427  * La máquina evalúa las cuatro direcciones posibles y selecciona
428  * el movimiento válido que minimiza la distancia Manhattan entre
429  * la posición resultante del robot y la salida.
430  *
431  * Si ninguna dirección es válida, el robot permanece en su posición.
432  */
433 public void goodMove() {
434     int[] pos = robot.coordinates();
435     int currentX = pos[0];
436     int currentY = pos[1];
437     char bestDirection = robot.direction();
438     int bestDistance = Integer.MAX_VALUE;
439
440     // Moverse hacia abajo
441     int newX = currentX;
442     int newY = currentY + 1;
443     if (inside(newX, newY) && !hasWall(currentX, currentY, newX, newY)) {
444         int distance = manhattanDistance(newX, newY, exitX, exitY);
445         if (distance < bestDistance) {
446             bestDistance = distance;
447             bestDirection = 'B';
448         }
449     }
450     newX = currentX;
451     newY = currentY - 1;
452     if (inside(newX, newY) && !hasWall(currentX, currentY, newX, newY)) {
453         int distance = manhattanDistance(newX, newY, exitX, exitY);
454         if (distance < bestDistance) {
455             bestDistance = distance;
456             bestDirection = 'N';
457         }
458     }
459     newX = currentX;
460     newY = currentY;
461     if (inside(newX, newY) && !hasWall(currentX, currentY, newX, newY)) {
462         int distance = manhattanDistance(newX, newY, exitX, exitY);
463         if (distance < bestDistance) {
464             bestDistance = distance;
465             bestDirection = 'S';
466         }
467     }
468     newX = currentX;
469     newY = currentY;
470     if (inside(newX, newY) && !hasWall(currentX, currentY, newX, newY)) {
471         int distance = manhattanDistance(newX, newY, exitX, exitY);
472         if (distance < bestDistance) {
473             bestDistance = distance;
474             bestDirection = 'E';
475         }
476     }
477     if (bestDistance != Integer.MAX_VALUE) {
478         turn(bestDirection);
479         move();
480     }
481 }

```

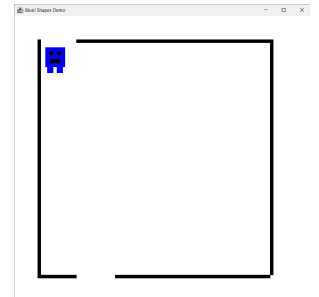
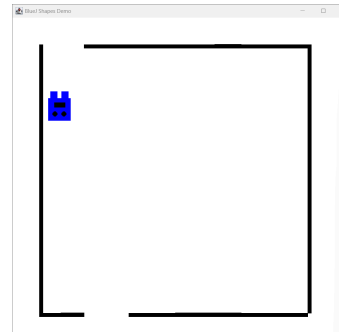


El robot usando goodMove() se va acercando a la entrada

```

/**
 * Deshace el último movimiento realizado por el robot.
 *
 * Recupera la posición y dirección anteriores almacenadas
 * en el historial. El uso de este método no consume vidas.
 *
 * Si no existe ningún movimiento anterior, el robot
 * permanece en su posición actual.
 */
public void undo() {
    if (!history.empty()) {
        RobotState previous = history.pop();
        robot.setPosition(previous.getX(), previous.getY());
        robot.turn(previous.getDirection());
        robot.setOK(true);
    }
}

```



Aplicando undo() regresamos el robot a la posición y dirección anterior

RETROSPECTIVA

1. ¿Cuál fue el tiempo total invertido en el laboratorio por cada uno de ustedes? (Horas)
Cada uno de nosotros invirtió aproximadamente 11 horas en el desarrollo del laboratorio.

2. ¿Cuál es el estado actual del laboratorio? ¿Por qué?

El laboratorio se encuentra finalizado, ya que logramos completar todas las actividades y funcionalidades propuestas.

3. Considerando las prácticas XP del laboratorio, ¿cuál fue la más útil? ¿Por qué?

La práctica XP Pair Programming fue la más útil, ya que mientras uno de nosotros programaba, el otro revisaba el código y ayudaba a identificar posibles errores. Esto nos permitió trabajar de manera más colaborativa y encontrar soluciones con mayor facilidad.

4. ¿Cuál consideran que fue el mayor logro? ¿Por qué?

Consideramos que nuestro mayor logro fue desarrollar RobotMaze, ya que fue una de las partes que más se nos dificultó debido a algunos métodos y a la lógica que debíamos implementar. A pesar de las dificultades, logramos comprenderla y hacer que funcionara correctamente.

5. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?

Nuestro mayor problema técnico fue que todavía teníamos conocimientos limitados de Java, especialmente porque en esta parte del laboratorio el nivel de dificultad era más elevado. Para resolverlo, investigamos, revisamos la documentación y utilizamos herramientas de inteligencia artificial como apoyo para comprender la lógica y encontrar posibles soluciones.

6. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?

Como equipo, consideramos que trabajamos bien al apoyarnos mutuamente y buscar soluciones juntos. Para mejorar nuestros resultados nos comprometemos a trabajar de manera más coordinada y a mantenernos informados sobre lo que cada integrante está realizando.

7. ¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados. Utilizamos las siguientes referencias:

- OpenAI. (2026). *ChatGPT*. <https://chatgpt.com/>
- Anthropic. (2026). *Claude*. <https://claude.ai/new>
- Oracle. (2026). *Java Documentation*. <https://docs.oracle.com/en/java/>

Las referencias que más nos ayudaron fueron ChatGPT y Claude, ya que nos permitieron comprender la lógica de algunos métodos que no sabíamos cómo implementar. Sin embargo, también utilizamos la documentación oficial de Java para consultar información sobre las funciones y características del lenguaje.